1. Características del lenguaje y PHP básico

Sitio:	Centros Granada 26	Imprimido por:	Triay Orfila, Marc
Curso:	Desarrollo web en entorno servidor	Día:	lunes, 5 de octubre de 2026, 17:10
Libro:	1. Características del lenguaje y PHP básico		

Tabla de contenidos

1. ¿Qué es PHP?

2. Sintaxis de PHP

3. Generando contenido

4. Comentarios

5. Variables
5.1. Declaración de variables
5.2. Asignación por copia y por referencia
5.3. Variables de variables
5.4. Variables no inicializadas
5.5. Variables predefinidas
5.6. Funciones de variables

6. Tipos de datos
6.1. Conversiones de tipo
6.2. Otros tipos de datos

7. Constantes

8. Operadores
8.1. Aritméticos
8.2. Lógicos
8.3. De asignación
8.4. Ternarios
8.5. Unarios
8.6. De asignación combinados
8.7. De ejecución
8.8. De comparación
8.9. De incremento/decremento
8.10. Bit a bit
8.11. De coalescencia nulo
8.12. Nave espacial
8.13. Nullsafe
8.14. Para cadenas
8.15. Para arrays
8.16. Precedencia de los operadores

9. Estructuras de control
9.1. Estructuras condicionales
9.2. Estructuras de repetición
9.3. Finalizar la ejecución de un programa
9.4. Otras estructuras de control
9.5. Sintaxis alternativa de las estructuras de control

10. Variables superglobales $_GET y $_POST

11. Excepciones y errores

11.1. Excepciones

11.2. Errores

11.3. Ejemplos excepciones

12. Mapa conceptual

1. ¿Qué es PHP?

PHP es un lenguaje de programación para desarrollo web en el lado del servidor. Es flexible y permite programar pequeños scripts con rapidez.

Se puede comparar con otros lenguajes de script que se ejecutan según el mismo principio: ASP (*Active Server Pages*), JSP (*Java Server Pages*) o PL/SQL Server Pages (PSP).

A diferencia de un lenguaje como JavaScript, donde el código se ejecuta del lado del cliente (en el navegador), el código PHP se ejecuta del lado del servidor. El resultado de esta ejecución se incrusta en la página HTML, que se envía al navegador. Este último no tiene conocimiento de la existencia del procesamiento que se ha llevado a cabo en el servidor.

Esta técnica permite realizar páginas web dinámicas cuyo contenido se puede generar total o parcialmente en el momento de la llamada de la página, gracias a la información que se recopila en un formulario o se extrae de una base de datos.

¿Qué puede hacer PHP?

2. Sintaxis de PHP

El elemento básico en PHP es el bloque. El bloque PHP está delimitado por las etiquetas correspondientes, y contiene una o varias sentencias separadas por punto y coma.

El código PHP se introduce dentro del HTML utilizando la etiqueta **<?php** para abrir el bloque de PHP y la etiqueta **?>** para cerrarlo. Las sentencias se separan mediante **;**

```
<?php
    Sentencia1;
    SentenciaN;
?>
```

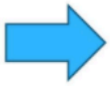
Además, cuando un fichero contiene solo PHP se recomienda no cerrar la etiqueta del último bloque. Puede dar problemas si la respuesta del servidor involucra a varios ficheros.

Por ejemplo, cuando codifiquemos clases o interfaces, sólo pondremos la etiqueta de apertura, para así indicar que es una archivo de PHP puro.

El servidor sustituye los bloques de PHP por su salida.

HTML y PHP

```
<!DOCTYPE  html>
<html>
  <head>
    <title>Hola  mundo</title>
  </head>
  <body>
    <?php
     echo  "Hola  mundo";
    ?>
  </body>
</html>
```



```
<!DOCTYPE  html>
<html>
  <head>
    <title>Hola  mundo</title>
  </head>
  <body>
    Hola mundo
  </body>
</html>
```

Los ficheros PHP deben tener extensión ".php"

3. Generando contenido

Tenemos varias posibilidades a la hora de generar contenido en nuestros documentos PHP, o lo que es igual imprimir texto en la pantalla:

- echo expresión;
- print(expresión);
- <?=expresión ?>

En general usaremos **echo** y **print** cuando lo hagamos dentro de un bloque de instrucciones y **<?=** cuando sólo vayamos a mostrar el valor de una variable dentro de un fragmento HTML.
En cualquier caso el resultado para una cadena de texto será el mismo:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Echo y print</title>
</head>
<body>
<p><?php echo "Este texto se mostrará en la página web." ?></p>
<p><?= "Este texto se mostrará en la página web." ?></p>
<p><?php print("Este texto se mostrará en la página web.") ?></p>
</body>
</html>
```

Puedes consultar sobre las diferencias entre echo y print tanto en el manual de PHP como en el siguiente artículo

4. Comentarios

Se puede utilizar comentarios:

- De bloque, encerrados entre "/*" y "*/"
- De línea, comenzando por "//" o por "#"

```
<!DOCTYPE html>
<html>
<body>

<?php
/*
This is a multiple-lines comment block
that spans over multiple
lines
*/

// This is a single-line comment

# This is also a single-line comment
?>

</body>
</html>
```

5. Variables

Una de las características principales de PHP es que es un lenguaje no fuertemente tipado. Esto quiere decir que no es necesario indicar el tipo de dato al declarar una variable. De hecho, las variables no se declaran, se crean la primera vez que se les asigna un valor.

El tipo de dato depende del valor con que se inicialicen. Esto agiliza la escritura de programas, pero también tiene inconvenientes. Si no se presta atención, puede dar lugar a código de baja de calidad y, a medida que las aplicaciones crecen, pueden darse errores difíciles de depurar.

5.1. Declaración de variables

En PHP los identificadores de las variables van siempre precedidos por el carácter '**$**'. Y son *case sensitive*: `$var != $vAR`

El identificador de la variable debe comenzar por una letra o un guion bajo ('_'), y puede estar formado por números, letras y guiones bajos.

Para declarar una variable, solo hay que asignarle un valor: **`$nombre = valor;`**

Por ejemplo, esta sentencia declara la variable $entero, que será de tipo integer porque se inicializa con un valor entero: **`$entero = 4;`**

También **es posible cambiar el tipo de dato** de una variable simplemente asignándole un valor de otro tipo de dato, como se puede ver en el siguiente ejemplo (líneas 8-12).

```
1  <?php
2  /* declaración de variables */
3  $entero = 4; // tipo integer
4  $numero = 4.5; // tipo coma flotante
5  $cadena = "cadena"; // tipo cadena de caracteres
6  $bool = TRUE; //tipo booleano
7  /* cambio de tipo de una variable */
8  $a = 5; // entero
9  echo gettype($a); // imprime el tipo de dato de a
10 echo "<br>";
11 $a = "Hola"; // cambia a cadena
12 echo gettype($a); // se comprueba que ha cambiado
```

La salida del ejemplo confirma que la variable cambia de tipo de dato.

El ejemplo utiliza la función **gettype()** que nos permite consultar el tipo de dato de una variable.

Conviene inicializar las variables para que no se produzcan errores.

5.2. Asignación por copia y por referencia

En principio, la asignación de variables se realiza mediante copia. Es decir, si hacemos:

```
$a = $b;
```

se crea una nueva variable a y se le asigna el valor que tenga b. Las variables a y b representan posiciones diferentes de memoria, aunque tengan el mismo valor después de la asignación.

También es posible definir una referencia a una variable utilizando el operador ampersand:

```
$var2 = &$var1;
```

En este caso $var2 no es una nueva variable con el valor de $var1. Por el contrario, $var2 apunta a la misma dirección de memoria que $var1, de manera que $var1 y $var2 son en realidad dos nombres para el mismo dato.

A continuación un ejemplo:

```
<?php

$a = 5;   // Variable $a tiene el valor 5
$b = &$a; // Se asigna por referencia, ahora $b y $a
apuntan al mismo valor

$b = 10;  // Cambiamos el valor de $b

echo "Valor de \$a: $a";  // Imprime: Valor de $a: 10
echo "Valor de \$b: $b";  // Imprime: Valor de $b: 10

?>
```

5.3. Variables de variables

Las variables de variables nos permiten establecer y usar dinámicamente los nombres de las variables.

Ejemplo:

```php
<?php
$a = 'hola';
$$a = 'mundo';
//las dos líneas siguientes producen la misma salida
echo "$a {${$a}}  ";

echo "$a $hola ";
```

Al poner el símbolo $ delante de $a se reserva otro espacio en memoria para alojar el nuevo contenido. Lo que hace es tratar la palabra “hola” como una variable.

Para acceder al valor de una variable de variable en una cadena de caracteres con dobles comillas, hay que encerrarla entre llaves ({}).

5.4. Variables no inicializadas

En versiones anteriores de PHP Si se intenta utilizar una variable antes de asignarle un valor, se genera un error de tipo E_NOTICE, pero no se interrumpe la ejecución del script. Si una variable no inicializada aparece dentro de una expresión, dicha expresión se calcula tomando el valor por defecto para ese tipo de dato. En este ejemplo podemos ver lo que ocurre al utilizar una variable no inicializada dentro de una expresión.

```
1  <?php
2  $var1 = 100;
3  $var3 = 100 + $var2;//$var2 no existe, se toma como 0
4  echo "$var3 <br>"; // muestra 100
5  $var3 = 100 * $var2; // $var2 no existe, se toma como 0
6  echo "$var3 <br>"; // muestra 0
```

En la línea 3, se intenta sumar una variable no inicializada. Como el valor por defecto para un entero es 0, el resultado de la suma es 100. En la línea 5, se intenta multiplicar por una variable no inicializada y, al multiplicar por 0, el resultado es 0.

La salida muestra un mensaje de error por cada intento de utilización de una variable no inicializada.

En la versión 8.5 que usaremos este curso si ponemos el código anterior nos daría un error de este tipo:

Warning: Undefined variable $var4 in /var/www/projects/u2-phpbasico/public/ejemplos/03-variables/3_no-inicializadas.php on line *6*

Call Stack

#	Time	Memory	Function	Location
1	0.0177	430456	{main}( )	.../3_no-inicializadas.php:0

100

Warning: Undefined variable $var4 in /var/www/projects/u2-phpbasico/public/ejemplos/03-variables/3_no-inicializadas.php on line *8*

Call Stack

#	Time	Memory	Function	Location
1	0.0177	430456	{main}( )	.../3_no-inicializadas.php:0

0

Para solucionar el problema:

a) Inicializar siempre las variables

b) Preguntar antes de asignar la variable. Ejemplo:

```php
// ejemplo para preguntar antes de usarla si existe y si no es así asignarle el valor 0
$var4 = $var4 ?? 0;

echo " <br> este cálculo no produce errores: ";
$var3 = 103 + $var4;
echo "$var3  ";
```

5.5. Variables predefinidas

Contienen información sobre el servidor, datos enviados por el cliente o variables de entorno. Para acceder a esta información, es necesario utilizar arrays asociativos. Dentro de las variables predefinidas hay un grupo de ellas, las superglobales, que están disponibles en cualquier ámbito. Las variables superglobales son relevantes para el desarrollo de aplicaciones web y las iremos conociendo a lo largo del curso.

Variables *superglobales*

Nombre	Descripción
$GLOBALS	Variables globales definidas en la aplicación
$_SERVER	Información sobre el servidor
$_GET	Parámetros enviados con el método GET (en la URL)
$_POST	Parámetros enviados con el método POST (formularios)
$_FILES	Ficheros subidos al servidor
$_COOKIE	*Cookies* enviadas por el cliente
$_SESSION	Información de sesión
$_REQUEST	Contiene la información de $_GET, $_POST y $_COOKIE
$_ENV	Variables de entorno

Para saber más acerca de ella consulta el manual de PHP

https://www.php.net/manual/es/language.variables.superglobals.php

5.6. Funciones de variables

Para manejar las variables existen en PHP numerosas funciones.

Su uso se tratará temas posteriores.

Entre ellas podemos destacar las funciones:

- **isset()** que nos indica si una variable existe y tiene un valor no nulo
- **empty()** nos indica si una variable está vacía . Es decir, si no existe o si su valor es igual a `false`.
- **unset()** para eliminar de la memoria una variable
- **is_integer(), is_string(), is_array(),** etc que devuelven true si el tipo que se pasa coincide con el tipo que indica la función
- **intval(), floatval(), strval()** que nos permiten convertir el tipo de una variable en otro diferente

Podemos encontrarlas todas en el manual de PHP.

6. Tipos de datos

Tipos de datos escalares:

Almacenan un dato atómico y unidimensional.

- **Booleano** (boolean). Sus posibles valores son true y false. Además, cualquier número entero se considera como true, salvo el 0 que es false. Este tipo de dato se obtiene, entre otros casos, como resultado de los operadores de comparación y se utiliza en sentencias condicionales y bucles.

Cuando se espera un valor boolean y se recibe otro tipo de dato, se aplican reglas de conversión.

Conversión implícita a *boolean*

Tipo	Valor como *boolean*
integer	Si es 0 se toma FALSE, en otro caso como TRUE
float	Si es 0.0 se toma FALSE, en otro caso como TRUE
string	Si es una cadena vacía o "0", se toma como FALSE, en otro caso como TRUE
variables no inicializadas	FALSE
null	FALSE
array	Si no tiene elementos se toma FALSE, en otro caso como TRUE

- **Entero** (integer). Cualquier número sin decimales, pudiendo ser positivo, negativo o el número cero. Se puede almacenar en varios formatos (base decimal, octal, hexadecimal y binario) aunque por defecto se almacena en numeración decimal.
- **Coma flotante** (float). Cualquier número real. Se pueden representar también en notación científica. Los redondeos puedan dar sorpresas.

Para que la salida sea correcta y puedas ver el número en el formato adecuado usa en lugar de la función echo la función printf.

Puedes comprobarlo con este ejemplo:

```php
<?php
$num_float = 34.0;
echo "La salida usando la función echo es: $num_float  <br>";
echo "Salida utilizando printf():";
printf(" %f ","$num_float" );
```

La conversión entre integer y float es automática. Aunque también se pueden usar los operadores de conversión (int) y (float).

No existe una precisión absoluta en el tratamiento de los números de punto flotante. El redondeo, comparación y cálculo tiene un error despreciable, pero si tenemos un número elevado de decimales debemos de aplicar las funciones matemáticas de precisión arbitraria de PHP. Ejemplos: bccomp(), bcmul(), bcadd(), etc

- **Cadena de caracteres** (string). Conjuntos de caracteres delimitados por comillas simples o dobles. Para delimitar una cadena se puede utilizar las comillas simples o dobles.

  Si se usan las dobles (*comillas mágicas*), las variables que aparezcan dentro de la cadena se sustituirán por su valor. Las comillas dobles son muy prácticas, ya que es más rápido insertar directamente las variables que montar las cadenas con varias concatenaciones.

  Existen algunos caracteres especiales que no pueden ser incluidos en una variable de tipo string a menos que indiquemos de alguna manera a PHP que son especiales para que pueda tratarlos de forma diferente Esto se hace con el símbolo de barra invertida(\) delante del carácter especial. Por ejemplo, para insertar comillas dobles en una cadena de texto acotada por comillas dobles tenemos que usar la secuencia de escape \.

Ejemplos:

```php
<?php
$mi_string = 'Puedo construir cadenas de caracteres con comillas
simples';
$mi_string2 = "y también puedo usar comillas dobles <br>";
$mi_nombre = "Ana";
echo $mi_string;
echo $mi_string2;
echo 'Debes tener cuidado si quieres incluir variables junto al
texto.';
echo 'Usando comillas simples NO podré escribir $mi_nombre';
echo "<br>Pero si puedo escribir $mi_nombre si uso las comillas dobles
<br>";
echo "También hay que prestar atención con las comillas \"dobles\"
dentro de otras dobles";
```

*A la hora de formatear la salida hay que tener en cuenta que va a ser procesada como HTML por un navegador web. Los caracteres de escape como "\n", "\r" y "\t" son ignorados por el navegador. Para introducir un salto de línea la opción más sencilla es la etiqueta HTML
.*

6.1. Conversiones de tipo

Si realizas una operación con variables de distintos tipos, ambas se convierten primero a un tipo común. Por ejemplo, si sumas un entero con un real, el entero se convierte a real antes de realizar la suma:

```
$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + $mi_real;
// La variable $resultado es de tipo real
```

Estas **conversiones de tipo**, que en el ejemplo anterior se lleva a cabo de forma automática, también se pueden realizar de forma forzada. Para hacerlo, escribiremos el nombre del tipo deseado entre paréntesis antes de al variable que se quiere forzar:

```
$mi_entero = 3;
$mi_real = 2.3;
$resultado = $mi_entero + (int) $mi_real;
// La variable $mi_real se convierte a entero (valor 2) antes de
sumarse.
// La variable $resultado es de tipo entero (valor 5)
```

En la documentación de PHP se especifican las conversiones de tipo posibles y los resultados obtenidos con cada una:

Conversiones de tipos posibles y los resultados obtenidos

6.2. Otros tipos de datos

- **Array**. Los arrays en PHP son una estructura muy flexible y potente. Sirven para representar colecciones de elementos con un único nombre.

  Unifica en un solo tipo lo que en otros lenguajes se consigue con arrays básicos, vectores, lista o diccionarios.

  Los elementos de un array se identifican por una clave, que puede ser un entero no negativo o una cadena. Por tanto, para acceder a los distintos valores de la variable se utiliza una clave (índice) que puede ser numérica o asociativa (alfanumérica).

  Los elementos guardan un orden dentro del array. Este orden está determinado por el orden de los elementos al declarar el array o al añadir nuevos.

  En PHP, los arrays no tienen que definirse de una forma concreta, sino que pueden tomar distintos tipos de valores: entero, objeto, etc. Simplemente puedes comenzar a asignarle valores.

  Tampoco es necesario decir de antemano el número máximo de valores que podrá tomar. Se podrán ir creando a medida que se necesitan.

  Ejemplo:

  ```
  <?php
  $miArray[1] = 45;// Asignación directa a un índice en concreto
  echo " Este es el primer array que hemos creado <br>";
  print_r($miArray);
  ```

  Una vez que hayamos estudiados funciones y estructuras de control veremos arrays con más profundidad.

- **Object**: Un objeto es una forma de definir de una forma abstracta un objeto del mundo real.

  Una clase define las propiedades y el comportamiento que tienen en común un grupo de objetos.

El tipo object se utiliza tanto como tipo de argumento como tipo de retorno, pero no puede usarse como tipo en la creación de una variable normal de forma explícita.

Un tipo object se crea usando la palabra reservada new.

Ejemplo:

```php
<?php

class Empleado
{
  private $nombre;
  private $apellidos;
  private $edad;

  public function __construct($nombre, $apellidos, $edad)
  {
    $this->nombre = $nombre;
    $this->apellidos = $apellidos;
    $this->edad = $edad;
  }
}
$miEmpleado = new Empleado('Carmen', 'Martín', 30);
```

Más adelante, profundizaremos en los conceptos de clase y objeto.

- **Callable: E**s un tipo de dato que representa **cualquier estructura que pueda ser invocada/ejecutada como una función** (utilizando paréntesis `$variable()`).
  Su utilidad principal radica en la **programación funcional y funciones de callback**: permiten pasar lógica personalizada como argumento a otra función para que esta la ejecute cuando sea necesario.

  Por ejemplo:

  ```
  function ejecutar(callable $funcion): mixed
  {
     return $funcion();
  }
  ```

  Formas de definir un callable:

- Funciones anónimas/Closures
- Funciones flecha
- Cadenas con el nombre de una función
- objetos invocables
- arrays para métodos de clases
- sintaxis de callable de primera clase

Ejemplo:

```php
function compara(mixed $valor1, mixed $valor2, callable $funcionComparacion): int
{
    return $funcionComparacion($valor1, $valor2);
}

// Callback para ENTEROS
$comparaEnteros = fn(int $v1, int $v2): int => $v1 <=> $v2;

//Callback para texto(Compara la longitud de la cadena)

$comparaLongitudTexto = fn(string $t1, string $t2): int => strlen($t1) <=> strlen($t2);

// --- Ejemplo 1: Comparando Números ---
$numero1 = 72;
$numero2 = 40;

$resultado = compara($numero1, $numero2, $comparaEnteros);

echo match ($resultado) {
    1 => "$numero1 es mayor que $numero2",
    -1 => "$numero2 es mayor que $numero1",
    default => "Los números son iguales",
};

echo '<hr>';
// --- Ejemplo 2: Comparando Cadenas por su Longitud ---
$texto1 = "Sistemas operativos";
$texto2 = "Programación";

echo match (compara($texto1, $texto2, $comparaLongitudTexto)) {
    1 => "'$texto1' es un texto más largo que '$texto2'",
    -1 => "'$texto2' es un texto más largo que '$texto1'",
    default => "Los dos textos tienen la misma longitud",
};
```

Esta función nos permite comparar dos valores , cualquiera que sea su tipo, pasando como argumento a la función el método de comparación. De esta forma, podríamos comparar con una única función tanto números como cadenas de caracteres. Nosotros le pasaremos la función específica para comparar los valores.

- **Null**. El tipo de dato null representa una variable que no ha sido asignada. Solo puede tomar un único valor, NULL. Se considera que una variable es de tipo null si se le asigna el valor NULL o no está inicializada. También puede utilizarse para indicar un valor por defecto nulo en argumentos de funciones o como valor opcional de retorno.

  Ejemplo:

```php
<?php
function hola(?string $nombre = null) {
    if (is_null($nombre)){
     echo '¡Hola!';
    }else {
     return '¡Hola '.$nombre.'!';
    }
}

hola();
hola('Marta');
```

- **Resource.** Este tipo de dato representa recursos externos, como una conexión a una base de datos, ficheros, etc. Más información…
- **Void.** En realidad, void sólo se permite como un tipo de retorno de una función, para indicar que no devuelve nada. Es decir la función realiza una o varias acciones pero no devuelve ningún tipo de información.
- **Unión.** Una variable de tipo unión indica que puede aceptar un valor de uno de los tipos indicados en la unión. El orden no es importante. La única restricción en cuanto a tipos es que no puede incluir el tipo void.

```php
class Number
{
    private int|float $number;

    public function setNumber(int|float $number): void {
        $this->number = $number;
    }

    public function getNumber(): int|float {
        return $this->number;
    }
}
```

- **Mixed**. El tipo mixed nos permite añadir tipos a los parámetros, propiedades de clase y retornos de función para indicar que no se ha olvidado la información del tipo, sino que simplemente no se puede especificar con más precisión o que explícitamente decidimos no hacerlo. Por ejemplo, para mantener la compatibilidad con versiones anteriores.

Equivale a un tipo *union* con la siguiente definición: *string| int | float| bool | null | array | object | callable| resource*

7. Constantes

Son tipos de datos que no varían durante la ejecución de un programa.

Para definir constantes se utiliza la función define(), que recibe el nombre de la constante y el valor que queremos darle. `define("LIMITE",1000);`

También se pueden definir constantes mediante la palabra reservada `const: const PI = 3.14;`

Definir constantes mediante la función define() no es lo mismo que hacerlo mediante la palabra reservada const. Las constantes definidas con define() se crean en tiempo de ejecución mientras que las constantes definidas con const se crean en tiempo de compilación. Eso significa que las primeras se pueden crear en cualquier momento durante la ejecución de un programa, mientras que las segundas no. Sin embargo, al definir una constante en una clase, se debe hacer con const.

Para mostrar el valor de las constantes en pantalla: `echo LIMITE;`

Si lo que deseamos es comprobar si una constante está definida o no usaremos la función defined():

```
<?php
define("TITULO", "Don Quijote de la Mancha");
if ( defined( "TITULO")) {
echo " El título del libro es :  ". TITULO;
}
```

Es habitual utilizar identificadores en mayúsculas para las constantes. Los nombres de las constantes deben empezar por una letra o guion bajo y tener sólo letras, números y guiones bajos.

PHP define de forma global varias constantes accesibles por los scripts llamadas **constantes predefinidas**. Algunos ejemplos son: PHP_VERSION , PHP_OS, PHP_INT_MAX , etc. En el manual puedes encontrar más **constantes predefinidas** del núcleo.

Las constantes que empiezan por un doble guion bajo(__) se llaman constantes mágicas y nos ayudan a depurar nuestras aplicaciones.

Ejemplo:

```php
<?php
    echo __LINE__ . " Version de PHP: " . PHP_VERSION. "<br>";
    echo __LINE__ . " Sistema operativo: " . PHP_OS . "<br>";
    echo 'Estoy en la línea: '.__LINE__ ."<br>";;
    echo 'El directorio en el que estoy: '.__DIR__;
```

En el ejemplo anterior con el segundo "echo" obtendremos exactamente el número de línea de código del script PHP y la versión del sistema operativo. Con el cuarto "echo" obtendremos la ruta completa del directorio del archivo en el que estamos trabajando.

Las **constantes predefinidas** o mágicas, en realidad ,no son constantes puesto que cambian dependiendo de donde se emplean.

8. Operadores

Los operadores son símbolos que sirven para hacer diferentes operaciones con los datos.

8.1. Aritméticos

Los operadores aritméticos son símbolo usados en matemáticas:

Ejemplo	Nombre	Resultado
+$a	Identidad	Conversión de $a a int o float según el caso.
-$a	Negación	Opuesto de $a.
$a + $b	Adición	Suma de $a y $b.
$a - $b	Sustracción	Diferencia de $a y $b.
$a * $b	Multiplicación	Producto de $a y $b.
$a / $b	División	Cociente de $a y $b.
$a % $b	Módulo	Resto de $a dividido por $b.
$a ** $b	Exponenciación	Resultado de elevar $a a la potencia $bésima.

8.2. Lógicos

PHP permite encadenar varias comparaciones juntas mediante este tipo de operadores:

Ejemplo	Nombre	Resultado
$a and $b	And (y)	**true** si tanto *$a* como *$b* son **true**.
$a or $b	Or (o inclusivo)	**true** si cualquiera de *$a* o *$b* es **true**.
$a xor $b	Xor (o exclusivo)	**true** si *$a* o *$b* es **true**, pero no ambos
! $a	Not (no)	**true** si *$a* no es **true**.
$a && $b	And (y)	**true** si tanto *$a* como *$b* son **true**.
$a || $b	Or (o inclusivo)	**true** si cualquiera de *$a* o *$b* es **true**.

8.3. De asignación

Los operadores de asignación se utilizan para dar valores a las variables:

$var = e1	Asignación **por valor**
$var2 = &$var2	Asignación **por referencia**
$var += e1, $var -= e1, $var *= e1, $var /= e1	Equivalentes a $var = $var + e1, $var = $var - e1, etc. Válido para cualquier operador binario aritmético de cadenas o arrays.

8.4. Ternarios

Este operador evalúa un operando y, dependiendo de si es verdadero o falso, evalúa el segundo operando o el tercero. La expresión que se quiere evaluar se escribe delante de un símbolo (?), después la expresión que tiene que ejecutarse si la evaluación anterior es true, seguida del símbolo (:) con la expresión que debe ejecutarse si es false.

En general: ***sentencia_booleana? valor_si_verdadero : valor_si_falso***

Usarlo nos ayuda a crear código más limpio y fácil de entender.

El operador ternario es una forma abreviada de la sentencia if-else, por lo que volveremos a hablar de él cuando veamos las estructuras condicionales.

```php
<?php
$a = 2 ;
$b = 1;
$resultado = $a > $b ? true : false;
echo($resultado);//true

// esto es equivalente al siguiente código
$a = 2 ;
$b = 1;
    if ($a > $b) {
        $resultado = true;
    } else {
        $resultado = false;
    }
    echo($resultado);//true
```

8.5. Unarios

El signo (-) se usa delante de un número o variable numérica . Tiene la propiedad de cambiar el signo del número al que se aplica. Este operador tiene la propiedad de cambiar el signo al número al que se le aplica.

8.6. De asignación combinados

PHP también dispone de una serie de operadores de asignación que permite realizar la operación en un único paso.

- *+=* Suma y asignación
- *-=* Resta y asignación
- **=* Multiplicación y asignación
- */=* División y asignación
- *%=* Módulo y asignación
- *&= Y*
- *| = O*
- *^= O exclusiva*
- *.=* Concatenación y asignación
- *>>= Desplazamiento a la derecha*
- *>> = Desplazamiento a la izquierda*

8.7. De ejecución

PHP ha adoptado la misma nomenclatura que se usa en Shell de Linux para ejecutar comandos del sistema. Los operadores de ejecución usan apóstrofo invertido sirve para ejecutar comandos del sistema.

Ejemplo:

```
<?php
$listado_archivos = `ls -al`; // Listado de todos los archiv
echo "<pre>$listado_arcivos<pre>"; // Se muestra en pantalla
```

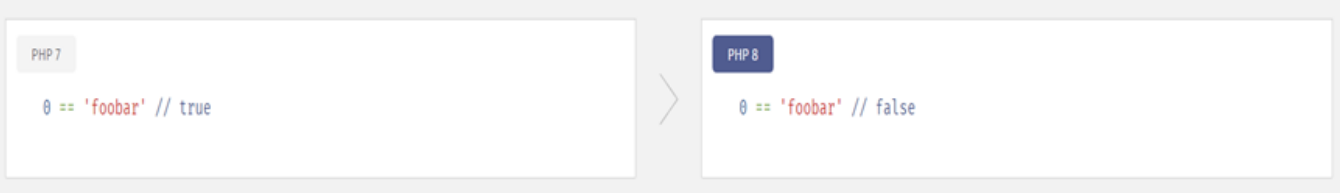
8.8. De comparación

Listado de operadores de comparación:

Ejemplo	Nombre	Resultado
$a == $b	Igual	**true** si *$a* es igual a *$b* después de la manipulación de tipos.
$a === $b	Idéntico	**true** si *$a* es igual a *$b*, y son del mismo tipo.
$a != $b	Diferente	**true** si *$a* no es igual a *$b* después de la manipulación de tipos.
$a <> $b	Diferente	**true** si *$a* no es igual a *$b* después de la manipulación de tipos.
$a !== $b	No idéntico	**true** si *$a* no es igual a *$b*, o si no son del mismo tipo.
$a < $b	Menor que	**true** si *$a* es estrictamente menor que *$b*.
$a > $b	Mayor que	**true** si *$a* es estrictamente mayor que *$b*.
$a <= $b	Menor o igual que	**true** si *$a* es menor o igual que *$b*.
$a >= $b	Mayor o igual que	**true** si *$a* es mayor o igual que *$b*.

Comparadores inteligentes:

En PHP 8 hay comparaciones inteligentes entre strings y números. En versiones anteriores a la 8 la evaluación de igualdad devuelve true aunque la cadena no contenga ningún carácter numérico. Esto sucede porque las versiones antiguas del algoritmo interpretan la ausencia de caracteres numéricos en un string como equivalente a 0, devolviendo este valor en la comparación. Este y otros fallos se corrigen con PHP8. Para saber más haz clic aquí.



8.9. De incremento/decremento

Estos son los operadores de incremento/decremento:

Ejemplo	Nombre	Efecto
++$a	Pre-incremento	Incrementa $a en uno, y luego retorna $a.
$a++	Post-incremento	Retorna $a, y luego incrementa $a en uno
--$a	Pre-decremento	Decrementa $a en uno, luego retorna $a.
$a--	Post-decremento	Retorna $a, luego decrementa $a en uno.

8.10. Bit a bit

Los operadores de **bit a bit** utilizan las variables a nivel bajo, tal y como se almacenan en memoria física y comparan bit a bit los valores.

Aquí puedes ver algunos ejemplos

8.11. De coalescencia nulo

El operador de **coalescencia nulo (fusión de null (??))** . Se utiliza cuando se necesita asignar un valor a una variable, dependiendo de si la variable de origen existe o no. Este operador devuelve el primer operando si existe y no es NULL; en otro caso, devuelve el segundo operador.

Ejemplo:

```
echo $variable ?? 'No existe';
```

Si la variable existe y no es nula se muestra su valor. En caso contrario se mostraría el mensaje 'No existe'.

Otro ejemplo:

```php
<?php

// Array que simula los datos de un usuario que provienen de una base de datos o formulario
$userData = [
    'first_name' => 'Carlos',
    // 'last_name' está ausente, simula un valor no definido
    'email' => 'carlos@email.com',
    // 'phone' es null, simulando que no se ha proporcionado el número de teléfono
    'phone' => null
];

// Valores predeterminados para los campos que podrían faltar
$default_last_name = "Desconocido";
$default_phone = "No disponible";

// Usamos el operador Null Coalescing para obtener el valor correcto o un valor predeterminado
$last_name = $userData['last_name'] ?? $default_last_name;
$phone = $userData['phone'] ?? $default_phone;

// Crear un mensaje de bienvenida
$welcome_message = "Bienvenido, {$userData['first_name']} {$last_name}.\n";
$welcome_message .= "Tu correo electrónico es: {$userData['email']}.\n";
$welcome_message .= "Tu teléfono es: {$phone}.";

// Mostrar el mensaje
echo $welcome_message;

?>
```

8.12. Nave espacial

El operador Nave espacial (<=>) se utiliza para comparar dos expresiones.

Las comparaciones se realizan según las reglas de comparación de tipos usuales de PHP.

Este operador devolverá -1, 0 o 1 si la primera expresión es menor, igual o mayor que la segunda expresión. Es un operador binario que realiza tres comparaciones.

Ejemplo:

```php
<?php
// Números enteros
echo 1 <=> 1; // 0
echo 1 <=> 2; // -1
echo 2 <=> 1; // 1

// Numeros decimales
echo 1.5 <=> 1.5; // 0
echo 1.5 <=> 2.5; // -1
echo 2.5 <=> 1.5; // 1

// Cadenas de caracteres
echo "a" <=> "a"; // 0
echo "a" <=> "b"; // -1
echo "b" <=> "a"; // 1
?>
```

Es muy útil cuando se escriben funciones de comparación definidas por el usuario usando usort, uasort o uksort .

Antes de que apareciera este operador se requerían expresiones más elaboradas con estructuras de control para conseguir lo mismo. Los objetos no se pueden comparar.

8.13. Nullsafe

El operador **Nullsafe** (?->) se utiliza para comprobar que una propiedad de un objeto o de un método no devuelve null. Cuando en PHP 7 queríamos hacer esta comprobación necesitabamos escribir varias sentencias if . Ahora con el operador nullsafe, si la primera propiedad, o el primer método, me devuelve null se anula la ejecución de toda la cadena. De esta forma se simplifica el código y se reduce la posibilidad de errores.

Ejemplo en PHP clásico:

```
$universidad = null;

if ($session !== null) {

    $usuario = $session->usuario;

    if ($usuario !== null) {
        $estudios = $usuario->getEstudios();

        if ($estudios !== null) {
            $universidad = $estudios->universidad;
        }
    }
}
```

Ejemplo en PHP 8:

```
$universidad = $session?->usuario?->getEstudios()?->universidad;
```

Si en cualquiera de las conexiones se rompe la cadena, el sistema automáticamente devolverá `NULL` y se detiene el proceso. Es decir, PHP comprobará si el primer operador **$session** da como resultado un valor nulo. Si no es así, la ejecución continuará hasta el último operador. PHP detendrá el proceso de ejecución cuando uno de los operadores resulte nulo.

Se ahorra tiempo y se favorece un código más limpio.

8.14. Para cadenas

Consulta en el manual los operadores para cadenas.

8.15. Para arrays

Consulta en el manual los operadores para arrays.

8.16. Precedencia de los operadores

La precedencia de operadores especifica que operaciones han de efectuarse primero en caso de existir más de una operación. En el manual de PHP puedes encontrar una tabla donde se enumera el orden de precedencia

9. Estructuras de control

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales y de repetición.

9.1. Estructuras condicionales

- if
- if-else Recuerda que el operador ternario es la forma abreviada de esta sentencia.
- if-elseif
- switch
- match.

Match se incluye a partir de PHP 8. Es parecida a switch, pero más legible, con más potencia y con la capacidad de devolver valores. Esta estructura recibe una variable y elige entre varias opciones la expresión que debe ejecutarse.

```php
<?php
$dia = 3;
echo match ($dia) {
    1=> "Hoy es lunes",
    2=> "Hoy es martes",
    3=> "Hoy es miércoles",
    4=> "Hoy es jueves",
    5=> "Hoy es viernes",
    6=> "Hoy es sábado",
    7=> "Hoy es domingo",
    default => " Ese día de la semana no existe",
};
```

9.2. Estructuras de repetición

- for
- while
- do-while

Dentro de un bucle se pueden usar las sentencias break y continue.

o Break: sirve para salir del bucle o switch en el que aparece y continúa el programa.

o Continue: Salta hasta el final de la interacción y continúa la ejecución desde el principio del bucle.

La sentencia continue fuerza una nueva iteración del bucle.

Las sentencias que estén dentro del bucle, pero después de continue no se ejecutan y, si se cumple la condición del bucle, se ejecuta una nueva iteración.

Si se trata de un bucle for, se ejecutan las instrucciones de autoincremento antes de evaluar la condición.

Un ejemplo de uso de continue y break con un bucle while:

```
<?php

    $contador = 0;

    while ($contador < 10) {
        $contador++;

        // Si el contador es 4, salta el resto de esta iteración
        if ($contador == 4) {
            echo "Saltando el número 4 con continue.<br>";
            continue; // Continúa con la siguiente iteración, no imprime el
            número 4
        }

        // Si el contador es 8, detiene el bucle por completo
        if ($contador == 8) {
```

```
            echo "Deteniendo el bucle en el número 8 con break.<br>";
            break; // Sale del bucle
        }

        // Imprime el valor del contador si no se ha usado continue
        echo "Contador: $contador<br>";
    }
?>
```

9.3. Finalizar la ejecución de un programa

Podemos parar la ejecución con las funciones exit() o die(). Las dos aceptan un parámetro que se imprime en pantalla.

9.4. Otras estructuras de control

Se puede organizar el código en diferentes scripts PHP según nos convenga y reutilizar ese código en otros scripts.

Para incluir otros ficheros utilizaremos las sentencias **include** y **require.**

```
include "mifichero.php";
```

```
require 'mifichero.php';
```

Se diferencian solo en el tratamiento de errores. Si no se encuentra el fichero especificado *include* genera un error del tipo E_NOTICE, pero *require* genera un error E_FATAL, lo que implica el fin de la ejecución del script. Si no hay errores, en ambos casos el fichero especificado se ejecuta. Si el fichero es una librería con funciones, estas se declaran y quedan disponibles para el fichero que lo incluye. Si el fichero contine sentencias fuera de una función, estas se ejecutan.

También están disponibles las sentencias **require_once** e **include_once**, análogas a las anteriores, pero con la diferencia de que solo se ejecutan los ficheros especificados si no han sido incluido o requeridos con anterioridad.

La sentencia include_once puede ser de ayuda para evitar problemas como la redefinición de funciones, reasignación de valores de variables, etc.

La inclusión de ficheros tiene una diferencia con otros lenguajes.

Supongamos que el fichero A incluye al fichero B. Las rutas relativas que aparezcan en B se interpretarán a partir del directorio de A.

Para solucionarlo, en el fichero B se utiliza dirname(__FILE__) que devuelve la ruta del fichero:

```
include ( dirname(__FILE__)."/".'fichero.php');
```

Por último, estudiaremos la sentencia **return** que finaliza la ejecución del fichero cuando se encuentra fuera de una función.

```
$a = 0;
if ($a === 0){
    return;
}
echo "Esto va detrás del if"; //no se ejecuta ya que antes se ha ejecutado el return
```

Si el script se estaba ejecutando mediante un require o include, la ejecución continúa en el fichero que lo incluyó. Si se pasa como argumento un número

entero, este se devuelve como valor de retorno.

Si se encuentra dentro de una función, finaliza la ejecución de esta y la función devuelve el argumento del return. La ejecución continua en el fichero o función que llama a la función.

9.5. Sintaxis alternativa de las estructuras de control

Algunas estructuras de control se pueden escribir sustituyendo la llave inicial por un símbolo de dos puntos y la llave de cierre por endif, endwhile, endfor o endswitch.

Esta sintaxis alternativa se suele utilizar cuando estamos trabajando en las vistas de un programa de PHP. Si estamos poniendo código incrustado es recomendable usar este tipo de sintaxis para hacerlo más visual.

10. Variables superglobales $_GET y $_POST

A menudo necesitaremos que se pueda establecer una comunicación entre el cliente y el servidor, probablemente interaccionando por medio de formularios cuyos campos tendrán que ser procesados para dar una respuesta adecuada.

El protocolo HTTP define varios métodos para establecer esa comunicación. Los más habituales son GET y POST.

El método POST se utiliza sobre todo para enviar formularios al servidor.

El método GET es el que se utiliza para solicitar página web a un servidor, por ejemplo, al seguir el vínculo o introducir una dirección a mano en la barra del navegador.

Cuando se usa el método GET se pueden pasar **parámetros** al servidor **en la URL**. Para ello, a la ruta normal para acceder a una página se le añade el carácter “?” como indicador de que empieza la lista de parámetros. Cada parámetro tiene su nombre, a la izquierda del igual, y un valor, a la derecha. Los argumentos están separados entre sí por el carácter ampersand.

Por ejemplo, se puede acceder con el navegador a: http://localhost/ejercicios-clase/saludo_personalizado.php?nombre=Ana&apellido=Prieto

Para acceder a los argumentos de la URL desde un bloque PHP se usa el array superglobal $_GET, que tiene un elemento por cada argumento presente en la URL. El nombre del argumento será la clave del elemento del array.

Para controlar si los parámetros se han pasado correctamente se pueden utilizar las funciones empty() o is_null(). Las dos devuelven TRUE cuando el parámetro no está presente en la URL. La diferencia está en los parámetros presentes, pero sin valor.

Para que los parámetros no aparezcan en la URL se usa el método POST. En el script al que se envía el formulario los parámetros están disponibles en el array superglobal $_POST. La clave de cada argumento dentro del array es el atributo name del elemento correspondiente en el formulario

Aquí puedes descargar un ejemplo de uso donde desde paso_variables_URL.php se pasan parámetros en la URL que son tratados en destino.php.

11. Excepciones y errores

El sistema de control de errores de PHP ha ido evolucionando a lo largo de las versiones. En el sistema básico, se generan errores de diferentes tipos, representados por un número. A partir de PHP 5 se introdujo un sistema de excepciones usando los bloques try/catch/finally. Y finalmente, en PHP 7, aparecieron las excepciones de clase Error.

Hay que tener en cuenta que, aunque podemos controlar los errores, es imposible recuperarse de cierto tipo de errores y con ello se detiene la ejecución del programa. Sin embargo, podemos recuperarnos de las excepciones. Utilizaremos las excepciones para cambiar el flujo normal de un script si ocurre un error concreto dentro de una condición (llamada excepción).

Excepciones

Para indicar que se ha producido un fallo o error en nuestro código se puede lanzar una **excepción**.

Una excepción puede ser lanzada ("thrown") y opcionalmente capturada ("catched").

Para controlar las excepciones se utilizan bloque **try/catch/finally.**

- El código susceptible de producir algún error se introduce en un bloque **try**.
- Cuando se produce algún error, se lanza una excepción utilizando la instrucción **throw.**
- Después del bloque try debe haber como mínimo un bloque catch encargado de procesar el error.
- Si una vez acabado el bloque try no se ha lanzado ninguna excepción, se continúa con la ejecución en la línea siguiente al bloque o bloques **catch**.
- Se puede añadir un bloque **finally** que se ejecuta después del try/catch, haya habido excepción o no.

try{

instrucciones;

} catch(Exception e){

instrucciones;

}finally {

instrucciones;

}

Por ejemplo, para lanzar una excepción cuando se produce una división por cero podrías hacer:

```
try {
        if( $divisor == 0)
            throw new Exception("División por cero.");
    }
catch (Exception $e) {
         echo 'Se ha producido el siguiente error: ' . $e->getMessage();
    }
```

Una vez que una excepción es atrapada en el bloque catch, el objeto *Exception* contiene el mensaje de error que fue lanzado usando la palabra clave *throw*. La variable $e del ejemplo anterior es una instancia de la clase *Exception,* por lo que tiene acceso a todos los métodos de esa clase. En este bloque debes definir tu propia lógica para el manejo de excepciones, es decir, qué es lo que quieres hacer exactamente con el error que atrapaste.

PHP ofrece una clase base Exception para utilizar como manejador de excepciones. Para lanzar una excepción no es necesario indicar ningún parámetro, aunque de forma opcional se puede pasar un mensaje de error (como en el ejemplo anterior) y también un código de error.

Entre los métodos que puedes usar con los objetos de la clase Exception están:

- getMessage. Devuelve el mensaje, en caso de que se haya puesto alguno.
- getCode. Devuelve el código de error si existe.

Hay una forma de centralizar el manejo de todas las excepciones no capturadas de forma que puedas controlar su salida desde un sólo sitio. Para

ello se utiliza set_exception_handler(), que establece una función de gestión de excepciones definida por el usuario.

Errores

En el sistema básico, ante determinadas condiciones (por ejemplo, utilizar una variable no inicializada) PHP genera un error. Hay diferentes tipos de errores, cada uno asociado con un número y una constante predefinida. Se puede controlar cómo se comporta PHP ante los errores mediante tres directivas del fichero php.ini:

· error_reporting: indica qué errores deben reportarse. Lo normal es utilizar E_ALL

· display_errors: señala si los mensajes de error deben aparecer en la salida del script (adecuado para desarrollo, pero no en producción)

· log_errors: indica si los mensajes de error deben almacenarse en un fichero (útil en producción)

· error_log: si la directiva anterior está activada, es la ruta en la que se guardan los mensajes de error

El valor de la directiva error_reporting es un número, pero para especificarlo lo habitual es utilizar las constantes predefinidas y el operador or a nivel de bit.

Errores y Registro

Valor	Constante	Descripción	Nota
1	**E_ERROR** (integer)	Errores Fatales en tiempo de ejecución. Éstos indican errores que no se pueden recuperar, tales como un problema de asignación de memoria. La ejecución del script se interrumpe.	
2	**E_WARNING** (integer)	Advertencias en tiempo de ejecución (errores no fatales). La ejecución del script no se interrumpe.	
4	**E_PARSE** (integer)	Errores de análisis en tiempo de compilación. Los errores de análisis deberían ser generados únicamente por el analizador.	
8	**E_NOTICE** (integer)	Avisos en tiempo de ejecución. Indican que el script encontró algo que podría señalar un error, pero que también podría ocurrir en el curso normal al ejecutar un script.	
16	**E_CORE_ERROR** (integer)	Errores fatales que ocurren durante el arranque incial de PHP. Son como un **E_ERROR**, excepto que son generados por el núcleo de PHP.	
32	**E_CORE_WARNING** (integer)	Advertencias (errores no fatales) que ocurren durante el arranque inicial de PHP. Son como un **E_WARNING**, excepto que son generados por el núcleo de PHP.	
64	**E_COMPILE_ERROR** (integer)	Errores fatales en tiempo de compilación. Son como un **E_ERROR**, excepto que son generados por Motor de Script Zend.	
128	**E_COMPILE_WARNING** (integer)	Advertencias en tiempo de compilación (errores no fatales). Son como un **E_WARNING**, excepto que son generados por Motor de Script Zend.	
256	**E_USER_ERROR** (integer)	Mensaje de error generado por el usuario. Es como un **E_ERROR**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
512	**E_USER_WARNING** (integer)	Mensaje de advertencia generado por el usuario. Es como un **E_WARNING**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
1024	**E_USER_NOTICE** (integer)	Mensaje de aviso generado por el usuario. Es como un **E_NOTICE**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
2048	**E_STRICT** (integer)	Habilítelo para que PHP sugiera cambios en su código, lo que asegurará la mejor interoperabilidad y compatibilidad con versiones posteriores de PHP de su código.	Desde PHP 5 pero no incluidoen **E_ALL** hasta PHP 5.4.0
4096	**E_RECOVERABLE_ERROR** (integer)	Error fatal capturable. Indica que ocurrió un error probablemente peligroso, pero no dejó al Motor en un estado inestable. Si no se captura el error mediante un gestor definido por el usuario (vea también set_error_handler()), la aplicación se abortará como si fuera un **E_ERROR**.	Desde PHP 5.2.0
8192	**E_DEPRECATED** (integer)	Avisos en tiempo de ejecución. Habilítelo para recibir avisos sobre código que no funcionará en futuras versiones.	Desde PHP 5.3.0
16384	**E_USER_DEPRECATED** (integer)	Mensajes de advertencia generados por el usuario. Son como un **E_DEPRECATED**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	Desde PHP 5.3.0
32767	**E_ALL** (integer)	Todos los errores y advertencias soportados, excepto del nivel **E_STRICT** antes de PHP 5.4.0.	32767 en PHP 5.4.x, 30719 en PHP 5.3.x, 6143 en PHP 5.2.x, 2047 anteriormente

La función error_reporting() permite cambiar el valor de la directiva error_reporting en tiempo de ejecución. De esta forma no hay que acceder al archivo php.ini

También es posible definir una función propia (personalizada) para que se encargue de los errores utilizando set_error_handler().

Excepciones error

Las excepciones de tipo Error no heredan de la claseException, así que para capturarlas hay que usar:

catch(Error $e){

...}

o, alternativamente , la clase Throwable, de la que se derivan tanto Error como Exception:

catch(Throwable $e){

...}

Aquí puede encontrar las excepciones Error predefinidas.

11.1. Excepciones

Para indicar que se ha producido un fallo o error en nuestro código se puede lanzar una excepción.

Para controlar las excepciones se utilizan bloque try/catch/finally:

o El código susceptible de producir algún error se introduce en un bloque try.

o Cuando se produce algún error, se lanza una excepción utilizando la instrucción throw.

o Después del bloque try debe haber como mínimo un bloque catch encargado de procesar el error.

o Si una vez acabado el bloque try no se ha lanzado ninguna excepción, se continúa con la ejecución en la línea siguiente al bloque o bloques catch.

o Se puede añadir un bloque finally que se ejecuta después del try/catch, haya habido excepción o no.

```
try{
instrucciones;
} catch(Exception e){
instrucciones;
}finally {
instrucciones;
}
```

Por ejemplo, podemos escribir el siguiente código para lanzar una excepción cuando se produce una división por cero:

```php
try {
    if ($divisor == 0)
        throw new Exception("División por cero");
    }
catch (Exception $e) {
    echo 'Se ha producido el siguiente error: '. $e->getMessage();
}
```

Para manejar las excepciones se utiliza la clase base Exception.

Para lanzar una excepción no es necesario indicar ningún parámetro, aunque de forma opcional se puede pasar un mensaje de error (como en el ejemplo anterior) y también un código de error.

Entre los métodos que puedes usar con los objetos de la clase Exception están:

- getMessage. Devuelve el mensaje, en caso de que se haya puesto alguno.
- getCode. Devuelve el código de error si existe.

11.2. Errores

En el sistema básico, ante determinadas condiciones (por ejemplo, utilizar una variable no inicializada) PHP genera un error.

Los errores en PHP se clasifican en tres grandes grupos:

- Fatal error (o crítico) son errores que pueden comprometer seriamente el funcionamiento del programa y por ello, para prevenir males mayores, se aborta la ejecución y el programa deja de funcionar por completo. Es el nivel de error más alto.
- Warnings (Advertencia) son menos severos que los errores fatales y no suponen la terminación del programa. El programa sigue su curso pero puede no funcionar correctamente o ser la causa de que más adelante se produzcan errores más graves.
- Notice (Aviso) son los errores de menor nivel de gravedad. Señalan la presencia de un posible error pero que podrían ocurrir también de forma normal en la ejecución del programa. La aparición de errores Notice no supone la terminación del programa y, aunque no suelen ser fuente de problemas serios, pueden hacer que el programa no funcione exactamente como deseamos.

Los diferentes tipos de errores están asociados con un número y una constante predefinida. Se puede controlar cómo se comporta PHP ante los errores mediante tres directivas del fichero php.ini:

- error_reporting: indica qué errores deben reportarse. Lo normal es utilizar E_ALL
- display_errors: señala si los mensajes de error deben aparecer en la salida del script (adecuado para desarrollo, pero no en producción)
- log_errors: indica si los mensajes de error deben almacenarse en un fichero (útil en producción)
- error_log: si la directiva anterior está activada, es la ruta en la que se guardan los mensajes de error

El valor de la directiva error_reporting es un número, pero para especificarlo lo habitual es utilizar las constantes predefinidas y el operador OR a nivel de bit.

Valor	Constante	Descripción	Nota
1	**E_ERROR** (integer)	Errores Fatales en tiempo de ejecución. Éstos indican errores que no se pueden recuperar, tales como un problema de asignación de memoria. La ejecución del script se interrumpe.	
2	**E_WARNING** (integer)	Advertencias en tiempo de ejecución (errores no fatales). La ejecución del script no se interrumpe.	
4	**E_PARSE** (integer)	Errores de análisis en tiempo de compilación. Los errores de análisis deberían ser generados únicamente por el analizador.	
8	**E_NOTICE** (integer)	Avisos en tiempo de ejecución. Indican que el script encontró algo que podría señalar un error, pero que también podría ocurrir en el curso normal al ejecutar un script.	
16	**E_CORE_ERROR** (integer)	Errores fatales que ocurren durante el arranque incial de PHP. Son como un **E_ERROR**, excepto que son generados por el núcleo de PHP.	
32	**E_CORE_WARNING** (integer)	Advertencias (errores no fatales) que ocurren durante el arranque inicial de PHP. Son como un **E_WARNING**, excepto que son generados por el núcleo de PHP.	
64	**E_COMPILE_ERROR** (integer)	Errores fatales en tiempo de compilación. Son como un **E_ERROR**, excepto que son generados por Motor de Script Zend.	
128	**E_COMPILE_WARNING** (integer)	Advertencias en tiempo de compilación (errores no fatales). Son como un **E_WARNING**, excepto que son generados por Motor de Script Zend.	
256	**E_USER_ERROR** (integer)	Mensaje de error generado por el usuario. Es como un **E_ERROR**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
512	**E_USER_WARNING** (integer)	Mensaje de advertencia generado por el usuario. Es como un **E_WARNING**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
1024	**E_USER_NOTICE** (integer)	Mensaje de aviso generado por el usuario. Es como un **E_NOTICE**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	
2048	**E_STRICT** (integer)	Habilítelo para que PHP sugiera cambios en su código, lo que asegurará la mejor interoperabilidad y compatibilidad con versiones posteriores de PHP de su código.	Desde PHP 5 pero no incluidoen **E_ALL** hasta PHP 5.4.0
4096	**E_RECOVERABLE_ERROR** (integer)	Error fatal capturable. Indica que ocurrió un error probablemente peligroso, pero no dejó al Motor en un estado inestable. Si no se captura el error mediante un gestor definido por el usuario (vea también set_error_handler()), la aplicación se abortará como si fuera un **E_ERROR**.	Desde PHP 5.2.0
8192	**E_DEPRECATED** (integer)	Avisos en tiempo de ejecución. Habilítelo para recibir avisos sobre código que no funcionará en futuras versiones.	Desde PHP 5.3.0
16384	**E_USER_DEPRECATED** (integer)	Mensajes de advertencia generados por el usuario. Son como un **E_DEPRECATED**, excepto que es generado por código de PHP mediante el uso de la función de PHP trigger_error().	Desde PHP 5.3.0
32767	**E_ALL** (integer)	Todos los errores y advertencias soportados, excepto del nivel **E_STRICT** antes de PHP 5.4.0.	32767 en PHP 5.4.x, 30719 en PHP 5.3.x, 6143 en PHP 5.2.x, 2047 anteriormente

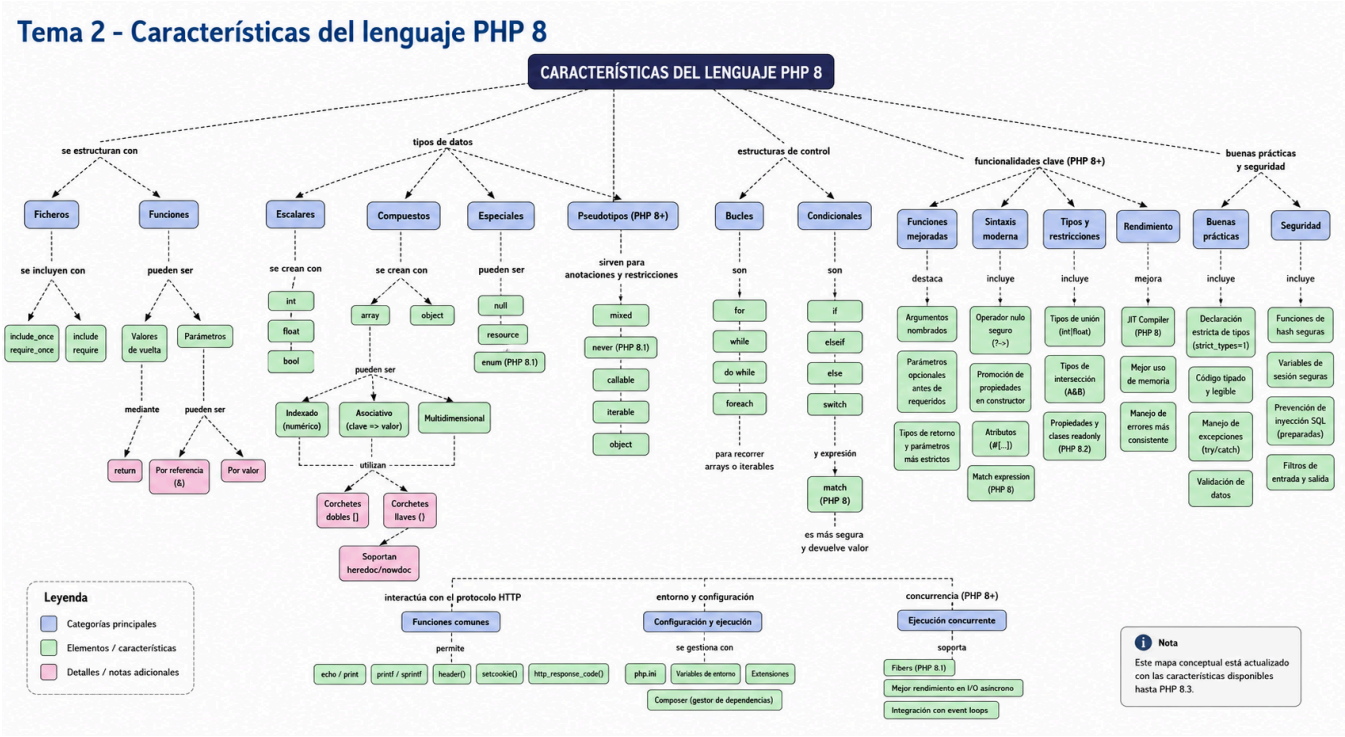
La función error_reporting() permite cambiar el valor de la directiva error_reporting en tiempo de ejecución.

También es posible definir una función propia para que se encargue de los errores utilizando set_error_handler().

11.3. Ejemplos excepciones

Descarga **aquí** los ejemplos

12. Mapa conceptual



Reiniciar tour para usuario en esta página